

Requirements: Python Prototype for GLORYS12V1 Ocean Reanalysis Visualization

Goal

Build a notebook-friendly Python prototype for visualizing high-resolution surface-ocean reanalysis data for the blueSteel ocean migration project.

The first dataset target is **GLORYS12V1**, using **local files**. The system should be designed so other datasets can be added later through configuration and dataset-specific adapters.

The first concrete demo is:

Render daily GLORYS12V1 sea-surface temperature over the North Pacific for June 1–10, 2018, as both a PNG snapshot and an MP4 movie.

Priorities

1. Correct scientific handling.
2. Dataset flexibility.
3. Notebook usability.
4. Visual polish and performance after the above are working.

Version 1 Scope

Version 1 should support:

- Local GLORYS12V1 files.
- Daily fields only.
- PNG snapshots.
- MP4 movies.
- Configurable region, projection, variable, date range, color scale, and output resolution.
- No track overlays in version 1.
- No automated data downloading in version 1.

Dataset Access

Create an abstract access layer that can later support:

- Local NetCDF files.
- Local Zarr stores.
- Remote OPeNDAP / THREDDS.
- Dataset APIs.

For version 1, implement **local files only**. Do not implement a downloader yet.

The data loader should:

1. Open local NetCDF or Zarr using `xarray`.
2. Support multiple files through `xarray.open_mfdataset`.
3. Subset by time and region before loading into memory.
4. Normalize dimension names where practical.
5. Use lazy loading with `dask`.
6. Handle longitude internally as **0-360 degrees**.
7. Handle North Pacific / Aleutian dateline-crossing maps cleanly.

Variables

Version 1 should support these ocean variables when available:

- Sea-surface temperature / surface temperature.
- Surface currents.
- Salinity.
- Mixed-layer depth.
- Sea-surface height.
- Chlorophyll.

Because not every dataset will include every variable, missing variables should produce a clear error message explaining which variable was requested and which variables are available.

For 3D variables, the surface definition must be **user-configurable by variable**. Supported selector modes should include:

- Top model layer.
- Nearest depth to a configured target depth.
- Explicit depth index.

Example:

```
variables:
  sst:
    source_var: thetao
    surface_selector:
      mode: nearest_depth
      depth_m: 0
  salinity:
    source_var: so
    surface_selector:
      mode: top_layer
  u_current:
    source_var: uo
```

```
surface_selector:  
  mode: top_layer  
v_current:  
  source_var: vo  
  surface_selector:  
    mode: top_layer
```

Derived Variables

Version 1 should include derived-variable support for:

1. Current speed from u/v components.
2. Front or gradient strength.
3. SST anomaly.
4. Thermal habitat masks.

Derived variables should be implemented through clear functions rather than hard-coded plotting logic.

Suggested derived variables:

```
current_speed = sqrt(u_current^2 + v_current^2)  
  
sst_gradient_strength = magnitude of horizontal SST gradient  
  
sst_anomaly = sst - configured climatology or baseline field  
  
thermal_habitat_mask = true where SST falls within configured temperature range
```

For SST anomaly, version 1 may support a simple baseline option first, such as subtracting a user-provided climatology file or a configured temporal mean. Do not attempt a full climatology pipeline unless needed for the first demo.

Thermal Habitat Bands

Include thermal habitat masks from the start.

Default bands:

- Broad steelhead surface-temperature envelope: **5–15°C**.
- High-use kelt envelope: **8–13°C**.

These should be configurable.

Example:

```
habitat_bands:
  steelhead_broad:
    variable: sst
    min: 5
    max: 15
    units: degC
  steelhead_high_use:
    variable: sst
    min: 8
    max: 13
    units: degC
```

Spatial Domain

The map region must be user-configurable.

Default region: broader North Pacific.

Use 0–360 longitude internally.

Suggested default extent:

```
region:
  name: north_pacific
  lon_min: 160
  lon_max: 235
  lat_min: 40
  lat_max: 68
```

The system must support regions crossing or near the dateline.

Time Handling

Version 1 supports daily fields only.

Movie rendering should:

- Use every available daily timestep.
- Render only actual available timesteps.
- Not interpolate missing dates.
- Skip unavailable dates only if configured to do so; otherwise warn clearly.

Visualization

Version 1 should produce **scientific / diagnostic** maps first, not polished presentation graphics.

Projection should be configurable, with a sensible North Pacific default.

Recommended default:

- Cartopy projection centered on the North Pacific / dateline.
- PlateCarree-compatible data transform.

Maps should include by default:

- Land.
- Coastlines.
- Colorbar with units.
- Date in title.
- Dataset name and variable name in title or subtitle.

Optional reference layers:

- Bathymetry or contours, if a bathymetry source is configured.

Do not include borders, labels, rivers, or other cartographic decoration by default.

Color Scales

Color scales should be fixed by variable by default, but configurable.

Example defaults:

```
color_scales:
  sst:
    vmin: 4
    vmax: 16
    cmap: turbo
    units: degC
  salinity:
    vmin: 30
    vmax: 34
    cmap: viridis
    units: psu
  current_speed:
    vmin: 0
    vmax: 1
    cmap: plasma
```

```
    units: m/s
ssh:
  vmin: -1
  vmax: 1
  cmap: balance
  units: m
mixed_layer_depth:
  vmin: 0
  vmax: 100
  cmap: viridis
  units: m
```

The user should be able to override `vmin`, `vmax`, and `cmap` in the config.

Outputs

Version 1 must produce:

- PNG snapshots.
- MP4 movies.

Do not require GIF output in version 1.

Output resolution should be configurable, with default **1080p**.

Movie frame PNGs should be deleted automatically after MP4 creation.

Filenames should be auto-generated and include:

- Dataset name.
- Variable name.
- Date or date range.
- Region name.
- Output type.

Example:

```
GLORYS12V1_sst_20180601_north_pacific_snapshot.png
GLORYS12V1_sst_20180601-20180610_north_pacific_movie.mp4
```

Track Data

Do not include steelhead track overlays in version 1.

Track overlay support should be treated as a future extension after ocean-field rendering is working.

Code Structure

Build a small reusable Python package plus example notebooks.

Notebook-friendly use is the primary interface for version 1. The CLI should exist, but notebooks should be the main user-facing workflow.

Suggested project structure:

```
ocean_viz/  
  configs/  
    glorys12_sst_demo.yaml  
  notebooks/  
    01_glorys12_sst_snapshot.ipynb  
    02_glorys12_sst_movie.ipynb  
    03_variable_smoke_tests.ipynb  
  outputs/  
ocean_viz/  
  __init__.py  
  config.py  
  dataset_loader.py  
  dataset_adapters.py  
  variables.py  
  derived.py  
  plotting.py  
  animation.py  
  cli.py  
  diagnostics.py  
  tests/  
    test_config.py  
    test_longitude.py  
    test_derived.py  
    test_smoke_dependencies.py  
  scripts/  
    run_snapshot.py  
    run_movie.py  
requirements.txt  
pyproject.toml  
README.md
```

Configuration

Use YAML for version 1. JSON support can be added later.

Example config:

```
project:
  name: blueSteel ocean reanalysis prototype

dataset:
  name: GLORYS12V1
  adapter: glorys12
  access:
    type: local
    path: /path/to/glorys12/files/*.nc
    engine: netcdf4

region:
  name: north_pacific
  lon_convention: 0_360
  lon_min: 160
  lon_max: 235
  lat_min: 40
  lat_max: 68

time:
  start: 2018-06-01
  end: 2018-06-10
  timestep: daily

variable:
  name: sst
  source_var: thetao
  display_name: Sea surface temperature
  units: degC
  surface_selector:
    mode: nearest_depth
    depth_m: 0

derived:
  enabled: false

visual:
  style: diagnostic
  projection: north_pacific_platecarree
  show_land: true
  show_coastline: true
  show_bathymetry: false
  color_scale:
    mode: fixed
    vmin: 4
    vmax: 16
    cmap: turbo
```



```
habitat_bands:
  enabled: true
  bands:
    - name: steelhead_broad
      min: 5
      max: 15
    - name: steelhead_high_use
      min: 8
      max: 13

output:
  output_dir: outputs
  resolution: 1080p
  dpi: 200
  auto_filename: true
  snapshot:
    enabled: true
    date: 2018-06-01
  movie:
    enabled: true
    format: mp4
    fps: 4
    delete_frames: true
```

Python Stack

Use:

```
xarray
dask
numpy
pandas
matplotlib
cartopy
netCDF4
zarr
pyyaml
imageio
imageio-ffmpeg
pytest
```

Optional:

```
cmocean
cf_xarray
```

```
bottleneck
geopandas
shapely
```

Assume development on **Mac** using `pip` and `virtualenv`.

Include both:

- `requirements.txt`
- `pyproject.toml`

Assume `ffmpeg` is already installed, but provide a smoke test that verifies it is available and compatible.

Dependency Smoke Test

Create a test or diagnostic command that checks:

1. Python version.
2. Required packages import successfully.
3. `cartopy` can create a simple map.
4. `xarray` can open a small sample NetCDF file or synthetic dataset.
5. `ffmpeg` is callable.
6. MP4 writing works on a short synthetic 2-frame animation.

Suggested command:

```
python -m ocean_viz.cli check
```

Public API

Provide notebook-friendly functions:

```
from ocean_viz import load_ocean_data, render_snapshot, render_movie

ds = load_ocean_data("configs/glorys12_sst_demo.yaml")
png_path = render_snapshot("configs/glorys12_sst_demo.yaml")
mp4_path = render_movie("configs/glorys12_sst_demo.yaml")
```

Provide CLI commands:

```
python -m ocean_viz.cli check
python -m ocean_viz.cli snapshot configs/glorys12_sst_demo.yaml
python -m ocean_viz.cli movie configs/glorys12_sst_demo.yaml
```

Required Implementation Details

Dataset loading

- Open local GLORYS12V1 files lazily.
- Standardize coordinate access for time, latitude, longitude, and depth.
- Convert longitudes to 0–360 internally.
- Sort longitudes after conversion.
- Subset before computing.
- Support dateline-safe region selection.
- Select surface layer according to config.
- Return an `xarray.DataArray` or `xarray.Dataset`.

Plotting

- Use `matplotlib` and `cartopy`.
- Use consistent color scales for all movie frames.
- Use a North Pacific-centered projection by default.
- Add land, coastlines, colorbar, title, and date.
- Save PNG at configured resolution.

Animation

- Render each daily timestep as a frame.
- Use consistent map extent and color scale.
- Encode MP4 using `ffmpeg`.
- Delete temporary frame PNGs after successful MP4 creation.
- Return final MP4 path.

Tests

Include smoke tests plus a small `pytest` suite.

Minimum tests:

1. YAML config loads and validates.
2. Longitude conversion to 0–360 works.
3. Dateline-safe region selection works.
4. Surface selector works for top layer and nearest depth.
5. Current speed calculation works.
6. Thermal habitat mask calculation works.
7. Dependency check confirms required packages and `ffmpeg`.

8. Synthetic snapshot renders.
9. Synthetic short MP4 renders.

Acceptance Criteria

Version 1 is complete when it can:

1. Run `python -m ocean_viz.cli check` successfully on Mac.
2. Load local GLORYS12V1 files.
3. Render a PNG snapshot of SST over the North Pacific for 2018-06-01.
4. Render an MP4 movie of daily SST over the North Pacific for 2018-06-01 through 2018-06-10.
5. Use fixed SST color limits by default.
6. Handle 0–360 longitude and dateline-centered North Pacific plotting cleanly.
7. Delete intermediate movie frames after MP4 creation.
8. Run the example notebooks successfully.
9. Pass the small pytest suite.

Non-Goals for Version 1

Do not implement:

- Automated Copernicus downloading.
- Remote API access.
- Steelhead track overlays.
- Interactive web maps.
- 3D globe rendering.
- CMIP future scenario processing.
- Bias correction or downscaling.
- Full climatology pipeline.
- Production data catalog management.